

Roblox Take Home Test Dev Log

Sundeep Singh

July 26, 2026

Contents

1	Foreword	3
2	Problem Statement	4
3	Naive Implementation	5
4	An Optimized Solution	7
4.1	Data Representation	7
4.2	Algorithm Optimization	8
4.3	Comparing to Known Solutions	12
4.4	Conclusions	12
5	The 4 Billion Extension	13
5.1	The Hopelessness of Iterating 4 Billion Times	13
5.2	Counting Algorithm	14
6	Answers	16
7	Algorithm Analysis	17
7.1	Space and Time Complexity Prediction	17
8	Cross Comparisons	19
8.1	When To Use Which	19
9	Time Comparison Plot	20
10	Future Work / Improvements	21
11	Bonus: The 4 Trillionth Number	22

1 Foreword

This document is part of my submission for the take home test. A LaTeX format was chosen because this is primarily a math problem, so I foresee that I will need to communicate math as clearly as possible. It will contain high level rationale and why certain decisions were made without diving into too much detail, but hopefully enough to follow my reasoning.

My approach for solving this problem from the outset is not indicative of how I would solve this problem in real production work. This problem reads like there are probably optimized solutions to this in existence already, and I would not try and reinvent the wheel. However, this is an interview, and I assume the goal is to see how I reason. In order to provide that information, I will not research existing methods until I get really stuck or cannot proceed. I will try and reason through the problem and arrive at the best solution I can myself first, pretending as if this is a bespoke problem with no known solutions.

This document is also not how I would record/present the results, it is a dev log. There will be all my critical thoughts and considerations that I have while reasoning about this problem. The code is also auto-clang formatted to match the google style.

2 Problem Statement

Consider a series in ascending order that only consists of numbers that can be factored into a product of 2, 3, and 5: 1,2,3,4,5,6,8,9,10,12,15...

The number 1 is included by definition.

The tasks are as follows:

1. Design and implement an algorithm to find the number that occupies the position 1500 in this sequence. (The answer is 859 963 392. Use this to verify the correctness of your algorithm.)
2. Improve your program to find the number at position 100 000. What is the problem with representing the result as a 32- or 64-bit integer? Find a more tractable number representation, don't print a decimal representation of the resulting number.
3. Attempt to optimize your program to find the number at position 4 000 000 000. (Only optimize your algorithm, no multi-threading or SIMD. If the program is unable to finish in a few minutes, discuss its limitations.)
4. Provide a detailed write-up, separate from your code, discussing the problem statement, algorithm and design choices, and the rationale behind your decisions. Additionally, discuss the results of your implementation, the time and memory complexity of your approach, and the associated trade-offs.

Note that task 4 is addressed by this document.

3 Naive Implementation

The problem here is about generating numbers that can be made up by multiplying together factors of 2, 3, and 5, and no other numbers (with 1 included as a definition). No other factors are allowed to be present in this number. I'll start with a brute force approach at first just to establish a correctness baseline. Let's get the correct answer first.

By definition then, any number in this sequence is representable as these three numbers multiplied together with certain powers:

$$N = (2^\alpha)(3^\beta)(5^\gamma) \tag{1}$$

In this formulation, note that the values $\alpha, \beta, \gamma = (0, 0, 0)$ represents the 1 that we need as the first entry, so that's nice. We get the 1 by definition popping out for free, we just add the origin to the possible search space.

Thinking about this further, since any number can be written down in this form, and the factors 2, 3, and 5 aren't expressible as multiples of each other, we can find if a number is valid just by removing the factors.

$$\frac{N}{2^\alpha} = 3^\beta 5^\gamma \tag{2}$$

But how do we find α ? A really naive approach is to just divide by 2 repeatedly until the number is no longer divisible. That gives us α .

Actually, we don't even need α . If we keep dividing by 2 until the remainder is non-zero, we'll have factored out the 2. Then we can repeat the process with 3, and then 5. This will repeat all the factors of 2, 3, and 5 out of the target number. And since the number can *only* be factorized by 2, 3, and 5, this process has to reduce to 1. If any other number is left over, there are other factors present.

I will try writing this idea in code and see if it works and gives the desired correct answer.

In this implementation, I was going to store the entire sequence, but I realize that we don't need to do that. If I just keep a counter that tracks how many valid answers I've found in the past, I can just return the value once that counter hits the desired target number. In an actual production setting, if we ever needed the sequence again, then it would be beneficial to store the sequence so we don't recompute it. But here, where we just need a one-off number, we

can save memory by only returning the singular value we need.

This naive approach does indeed give the correct answer, but it is very slow. Even for a modest number like 1500 entries, it took 14,217 ms. This will not scale to larger values.

Another thing to note is that even at a relatively small value of 1500 numbers, the number is already massive at 859 million. This means most numbers are invalid, and the valid numbers will quickly outpace the capacity of an unsigned integer to store for larger values. But for now, the naive implementation is complete, and question 1 is done.

4 An Optimized Solution

4.1 Data Representation

Clearly the naive method will not work or scale, it is very inefficient. We need something new.

The first problem to solve is the data representation. Moving from `int` to `uint64_t` seems to be the easiest solution so far. The maximum value a standard `int` can store is 2,147,483,647. This won't be large enough at the rate the values are scaling up, but `uint64_t` might be, with a maximum value that is around 18 quintillion.

However, given that I don't know the sequence of numbers, I actually have no idea if even that will be big enough. I considered scientific notation for a bit, store exponents of 10 and the number to be multiplied. But I realized that it doesn't work at all as most of the digits are non-zero in these numbers.

What if we used a number representation that is natural to the problem? In equation 1, I wrote the numbers down as powers of 2, 3, and 5. That seems like the natural basis for this problem. Instead of the entire number, let's represent the number as a triplet of unsigned integers:

$$N_f := (\alpha, \beta, \gamma) \tag{3}$$

Where the subscript means it is the factorization representation. Now if each exponent is stored as a `uint64_t`, that is a *massive* space of numbers we can store. I doubt we'll exhaust this. If we need to print the result in a user readable format, it'll be tricky, but let's cross that bridge when we come to it.

4.2 Algorithm Optimization

Clearly trying every single number is not good. I haven't benchmarked how much of the time is lost checking invalid numbers, but if by the number 860 million, we have only 1500 valid entries, that means that only 0.02% of numbers are valid. So we spent 99.98% of the time checking invalid numbers. That's a massive waste.

Let's focus instead on actually generating the numbers themselves instead of trial and error. Every number can be represented as a triplet, so numbers in the sequence can be generated by constructing increasing values of the exponents in their ranges. Kind of like binary counting.

The problem with this is that in binary counting, there is a max you count to before you flip the next bit. If we arrange the three exponents like "bits," how do we know when to stop counting one bit to increase the next?

Initially I thought I would just set the max to be the cube root of $n + 1$ (the plus 1 is because 0 is an allowed number), rounded up to the nearest integer. That way when I calculate the total number of numbers generated, I would get N numbers generated. However, I realized that this doesn't work.

Thinking back to the initial sequence of numbers. Say I was counting up the exponents, and I did 2^*2 . This is less than 5, which might be later in my counting sequence depending on how I ordered the exponents in my counting. Counting like bit flipping guarantees that I generate n unique valid numbers, not that I generate the *first* unique valid numbers in the ordered set.

I will need to overcount, but by how much? How can I guarantee that I've counted enough that I have the complete set of numbers up to n ? Say I add one more entry. If it is larger than the entry in the current n th position, sorting the result will place it after the n th entry. But if it is smaller, sorting the result will place it before the n th entry, thus changing the n th entry.

Say I have counted up to some value of the triplet (α, β, γ) . What is the smallest possible value I could generate next? I would need to increment each power by 1, and see which is the smallest. At first I thought you would always scale 2, but then I realized that this isn't the case as it's exponentials. If I am at $\gamma = 0$ and $\alpha = 200$, increasing γ would be the smaller increase, as adding the first multiple of 5 is smaller than adding the 201st multiple of 2. So:

$$N_{SM} = \min(2^{\alpha+1}3^{\beta}5^{\gamma}, 2^{\alpha}3^{\beta+1}5^{\gamma}, 2^{\alpha}3^{\beta}5^{\gamma+1}) \quad (4)$$

Where the subscript SM means the smallest missing number. We start at the already created values, and we build upon it.

After thinking a while, I realized that equation 4 is incorrect. This is building up from the largest known value so far. But, we have no numbers in the system that have $\alpha + 1$ in *any* position. That means that $\alpha + 1$ represents an entirely new unique number that we have not calculated yet, and is much smaller than my previous lower bound. So the actual real smallest possible missing value is:

$$N_{SM} = \min(2^{\alpha+1}, 3^{\beta+1}, 5^{\gamma+1}) \quad (5)$$

Equation 5 is the real smallest possible value that we missed, not equation 4.

Once the smallest possible missing value is greater than the entry we have in the n th position, we are guaranteed that the entries up until the n th position are complete.

Let's start by implementing this really naively. Just set the maximum values of the triplet to be consistent, and keep increasing the bounds uniformly until we hit the desired threshold. Correctness can be checked against the known naive algorithm. We will need to sort every iteration though, which will be a large cost.

In this very naive approach, where we just increment all the limits of the values by 1 uniformly, generate all possible values in the set fresh for each new bound increase and sort, and then check to see if the smallest possible missing value is greater than the n th sorted entry, the performance upgrade for the 1500th number is astonishing. This new algorithm can generate the number in 51 ms, compared to the previous 15k ms. That is a 300x improvement.

Generating 100k numbers now, I find the answer to be (in factorization representation):

$$N_{100,000} = 2^{96} 3^{15} 5^{13} \quad (6)$$

Note that this is about 20 orders of magnitude larger than the largest representable uint64_t number (verified using wolfram alpha). But also note how the values we actually store (the exponents) are much smaller and more tractable. Showing that we have indeed met the criteria for finding a good representation of these numbers.

Performance however is lacking. To calculate 100k iterations like this took 34,172 ms. This makes larger problems completely unviable. There are a few problems (that were knowingly made to get something that works first):

1. The bounds are increasing uniformly instead of intelligently
2. We are not re-using past generate numbers, each set is generated fresh
3. We sort the full sequence every bound increase

This is a good starting point for correctness, as it is easy to code and verify. But, we can do better.

We don't need to rebuild the entire sequence each time. As established before, each time we increment an exponent, that exponent has never been in a number before. So say we increase α , if we vary β, γ in their currently allowed range while fixing α at its incremented value, we will generate all new numbers that are not in the sequence. This solve two problems at once: we can reuse past constructions, and we can increase the bounds meaningfully. The bounds should be increased according to the smallest gain, same way that the smallest possible missed value was determined. This way, we are guaranteed to generate all the values without reusing any entries: always generate the smallest possible missing value and its combinations.

By implementing this idea, the time to calculate was cut down to about 9000 ms. This is about a 3.8x performance speedup. It's still too slow to make the next question possible. I am guessing that sort is what's taking so long, but I can do some micro-benchmarking and find out.

My benchmarking showed that 12 ms were spent generating the numbers, but 8500 ms were spent sorting. We don't need the full sorted list, maybe I can find a better way.

After spending a while and not making progress, I checked AI to see if there were any algorithms that will give me the Nth sorted element without sorting the entire array. Turns out there is, it is `std::nth_element`. I'm going to incorporate it.

This worked out wonderfully. First, I made an optimization to not even sort the list until there are at least n entries. Following this, using `std::nth_element` gave me the 100kth entry in the sequence in 645 ms, with only 386 ms of it spent sorting. This is now quite fast.

I realized that I had been building the debug build of the solution the entire time. In release mode, this algorithm now runs in 173 ms. This is fast enough

for this stage. The next step is now verifying the results.

4.3 Comparing to Known Solutions

Now that we've come up with a solution that is working to the best of our knowledge, let's find what the standard way of solving this problem is by doing some research. If there is a better way of doing things, we should test and use it.

After some research, turns out this problem is called the Ugly Numbers problem, and is fairly well known with well-established solution. The solution works by interleaving the different streams of numbers generated by repeated multiplications of the factors. Starting with 1 as the base, you apply each factor and pick the lowest result of that application. Whichever pointer's value was applied, increment that pointer, and check the remaining multiplication result again for each pointer location. This effectively interleaves different number streams obtained by repeatedly applying the factors. This also guarantees sort-ness without overgenerating, like my method did.

On trying to implement this, I noticed that the data representation issue is still a problem. Standard `uint64_t` cannot describe the solution, as I showed prior. My data representation work will still need to be mixed in.

Since this is a standard algorithm I used github copilot to accelerate the code writing for me while using my Triplet representation. What I find is that the standard method is a lot faster than mine, finishing the generation in 1 ms, compared to my 70 ms in the same run. However, my algorithm's results at least remain correct.

4.4 Conclusions

The bespoke method I came up with does indeed work and gives the correct answers. However, it is not the fastest method out there, and the standard stream interleaving method is at least a magnitude faster than my algorithm for the 100k entry case. While enlightening, in actual production cases I would recommend using the standard algorithm.

5 The 4 Billion Extension

5.1 The Hopelessness of Iterating 4 Billion Times

The final question is to extend this to the 4 billionth number in the the sequence. Immediately, there are some big problems here. Memory and time.

If we want to store 4 billion `uint64_t` values, that is 4 billion * 8 bytes = 32 GB. That is simply not feasible. All our previous methods have to be abandoned at this scale.

Before we do, let's say we had infinite memory and see how long the standard method would take. It takes 1,230,300 ns to do 100,000 iterations. The time cost on each iteration the same (scales as $O(n)$). So at this rate, it would take 1,230,000 ns * 4 billion / 100,000 $\approx$ 49.2 seconds. This means time is not the limiting issue here, but memory. 49.2 seconds is still slow, but at least it isn't an obscene amount of time.

What we need is a method of *predicting* the 4 billionth entry, not generating the entire sequence.

At this point, I am running low on the stated time cap. Given more time, I would have loved to play around with this and work on deriving an original method, but I don't have any concrete ideas on a predictive algorithm. I will therefore turn over to research and AI to help find an algorithm, and verify it.

5.2 Counting Algorithm

On research, I found there is a counting algorithm for this, that generates an estimate for how many ugly numbers are less than the 4 billionth entry, and then uses that to do a bisection-type search, and then narrow down the exact answer. The steps are as follows:

First, using the factorization formulation shown in equation 1, we can establish that the log of the number is

$$\log(N) = \alpha \log(2) + \beta \log(3) + \gamma \log(5) \quad (7)$$

Since log is a monotonically increasing function, we can work with the log function when counting. The idea is, if we can find some value X such that there are at least 4 billion ugly numbers less than it, we can use that to narrow down an area where the number lives, and then generate the sequence there to get the exact answer. First, for some number of X , we can compare the logs to find the highest exponents that result in answers still below X .

$$\alpha \log(2) + \beta \log(3) + \gamma \log(5) \leq X \quad (8)$$

In doing so, if we can find how many grid points α, β, γ lie below the plane defined in equation 8, that tells us how many ugly numbers are below X .

Since each unit cube in the search space is unit length, we know the count will be somewhat proportional to the volume. Since each cube has 4 vertices, this isn't the count, but it will be proportional to the count and so gives us a starting point.

I am running low on time, so I will skip some of the derivations here, but the idea is to generate a tet defined by the plane in equation 8, and the 3 coordinate planes, and compute it's volume. This is defined as

$$V_{tet} = \frac{L^3}{6 \log(2) \log(3) \log(5)} \quad (9)$$

Say now we want N numbers, we can set this equal to N to get an approximate log value:

$$L \approx (6N \log(2) \log(3) \log(5))^{1/3} \quad (10)$$

Now that we have a starting approximation for the log, we begin counting. Going back to the log formulation in equation 7, we can start with the largest base

and count how many powers we can have of it before it is larger than L . Once we have that set, we fix γ for each of those values, and then count how many powers of 3 we can add while staying under L . Once we have those, we can then fix those values of 3, and then count how many powers of 2. This counting problem will give us a number of ugly numbers less than the target number.

This number will be more or less than 4 billion. A binary search is then written until the L value starts to converge towards a value. Once the search region is small enough, we can enumerate exact representations in that region until we find our target number.

$$L_{low} < \alpha \log(2) + \beta \log(3) + \gamma \log(5) < L_{high} \quad (11)$$

If we know that the count of L_{low} is 20 less than N , then we enumerate 20 entries starting at L_{low} , and then we have the exact answer.

Due to time constraints, I'll be using copilot pretty heavily to help speed up the coding.

There are a few optimizations I wish I could make given more time, such as optimizing the size of the narrowed down shell, or figure out a better window sliding algorithm. But as for now, I have to leave it at the AI suggested initial passes.

The implementation works. Testing on the 100kth entry, we get consistent answers with the other two algorithms, but done in 187,000 ns as opposed to the standard method's 1,340,000 ns, or my bespoke method's 73,769,300 ns.

It does successfully run on 4 billion examples as well, giving the result:

$$N_{4,000,000,000} = 2^{375} 3^{2264} 5^{210} \quad (12)$$

And it did this in 1 ms. I have no correct answer to compare it against, but its correctness in earlier smaller examples makes me believe it is correct.

I do not claim to have come up with this. This was a really clever solution. I merely did research and used AI, and the algorithm that I got back made sense to me mathematically. Given quite a bit more time, I think maybe I could have come up with this myself, but the skill I'm demonstrating here is research, algorithm validation, and using AI to help accelerate development and iteration.

6 Answers

For the 3 stated problems, the answers I came up with are:

$$N_{1500} = 2^{17}3^85^0 \tag{13}$$

$$N_{100,000} = 2^{96}3^{15}5^{13} \tag{14}$$

$$N_{4,000,000,000} = 2^{375}3^{2264}5^{210} \tag{15}$$

7 Algorithm Analysis

7.1 Space and Time Complexity Prediction

For the naive number generation, every integer is repeatedly divided by the factors. A rough estimate for how many factors that is can be provided by the log:

$$\log(x_N) = \alpha \log(2) + \beta \log(3) + \gamma \log(5) \quad (16)$$

where the exponents correspond to how many times we had to divide each number. It also iterates over every number, not just the N numbers, so it's proportional to x_N as well, making the time order

$$O_{time}(naive) = O(x_N \log(x_N)) \quad (17)$$

The space complexity is quite nice, as all it does is store a counter.

$$O_{space}(naive) = O(1) \quad (18)$$

For my bespoke optimized solution (which wasn't so optimal in the end), there are a few time costs. Each number is generated once, so we have a $O(n)$ time contribution. Looking up the `nth_element` cost, it sorts in linear time. Conservatively, this is $O(n)$ as well, but realistically the size of the vector was smaller to start, and so it is likely less than this. Conservatively, I will say this is an upper bound, and give an overestimation of

$$O_{time}(bespoke) \leq O(N^2) \quad (19)$$

It's space cost is pretty straight forward, sitting at just storing all the entries. We did overgenerate, but it should just be some multiple of the target goal. So

$$O_{space}(bespoke) = O(N) \quad (20)$$

For the standard method, it's behavior is really easy to analyze. It did a single multiplication per iteration, and it only iterated N times. It also stored all the values, so space complexity is simple too.

$$O_{time}(standard) = O(N) \quad (21)$$

$$O_{space}(standard) = O(N) \quad (22)$$

For the predictive method, I looked up the time complexity and found it to be

$$O_{time}(predictive) = O(N^{2/3} \log(N)) \quad (23)$$

I want to reason through this, but I am very low on time. Research results will have to do here, I will verify on my own time.

The space complexity is relatively easy. We only generated the thin shell, so no matter how large the value is, we only store at most that defined value, so it's constant in space.

$$O_{space}(predictive) = O(1) \quad (24)$$

8 Cross Comparisons

8.1 When To Use Which

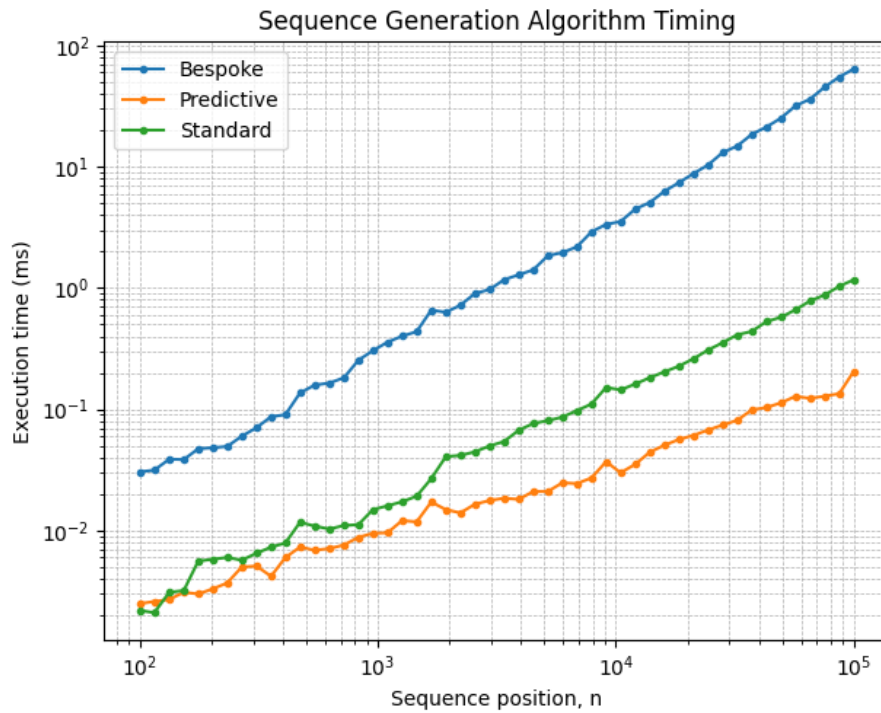
Given these results, I would say the following about the methods:

- The naive approach is only a demonstrative tool, not good for any actual applications.
- The bespoke solution serves as a good cross-check to standard methods. The reasoning is solid, it is decently performant, a good cross-validation tool.
- The standard three-pointer method is best used for most use cases. It is simple to reason, simple to debug, and performant for any real application ranges.
- The predictive model is the fastest across the board and the most capable across all ranks. However, it is very difficult to debug or reason about, or explain, and unless the standard method is found to be a bottle neck or massive values are needed, I would probably steer away from this solution in production settings.

9 Time Comparison Plot

I am actually out of time at this point and am over, but I want to provide some more concrete data here for decision making, as I would in an actual production setting. Here, I will use AI to generate some python scripts to plot the performance of the 3 main algorithms. I'm not a python developer, so I usually just use AI to generate these quick plots for me from output data.

Here is a comparison of the 3 main generation algorithms:



I was able to show that all of the algorithms agree for all of these numbers, so correctness is shown. But clearly, predictive is the standout method here. But for the reasons I outlined earlier, I would only implement it if the standard method really becomes a bottleneck.

It is important to note that across this entire range, all three of the algorithms agreed. A three-way verification is a great way of making sure the answers are correct.

10 Future Work / Improvements

The accompanying code submission is not ready for production work. There are many things still to improve and work on, such as:

- Optimization. The code so far is pure algorithmic optimization, there is no attempt at leveraging any sort of parallelization, SIMD, GPU acceleration, etc. Even minor optimizations like not having heavy constructors like the Triplet constructor inside loops. Triplet computes a log every time it's constructed. There is a lot of optimization still left to be done, currently we have only completed the correctness step.
- Code structure hygiene. This is not how I would structure actual production code. Everything lives in 2 coupled files, there are lots of lambdas that cannot be individually tested, common math operations could be split out into their own library and files, and many more refactorings. In the time limit, this structure felt appropriate, but this could definitely use refactoring.
- Deeper testing. My current tests are fairly shallow, just doing some sanity and agreement checks. I would ideally write a full unit test harness, and expand on the test suite quite a bit. I would also measure the memory consumption and explicitly plot the memory dependency of the algorithms. I would also fit the time and memory plots and extract explicit measurements of the order of the algorithms.
- Exploring third party libraries. I wonder if there are standard implementations of these algorithms out there. Or perhaps I could use boost or eigen to accelerate the code. This type of exploration would be good future work.
- Context analysis. This is anything I can do for this task, but for a general task I would also want to figure out why this code is needed, what are the contracts it needs to fulfill, and make sure that it meets all requirements from the outset of the development. But at this stage, I would reconfirm and test those requirements explicitly.
- Extend to generic factors. Instead of just limiting to powers of 2,3, and 5, we could extend this to any set of integers. Or maybe we want to allow factors of 7 to be present, but only if there is a matching factor of 5. As this is just a puzzle problem as far as I can tell, meaningful extensions aren't obvious, but we could explore extensions for actual production code.

There is a lot still to be done with this code, it's only a starting point.

11 Bonus: The 4 Trillionth Number

For fun, since our predictive algorithm is not memory limited, let's predict the 4 trillionth entry as well. Really put the algorithm through its paces. This is just a bonus, beyond requirements.

Running the predictive algorithm on 4 trillion yields:

$$N_{4 \times 10^{12}} = 2^{16,348} 3^{16,503} 5^{873} \quad (25)$$

This took 5.3614169 s. Still fairly fast, but getting slower.

I tried to run this on 4 quadrillion numbers, but it's been running for about 10 minutes now and still no result. If I find the number, I'll update it here!